

UPWORK PROPOSAL ASSISTANT

Database Design Document Version 2.0



Project Development Group Members

Sr. No.	Name	Registration Number
1	Tayyab Shehzad	NUM-BSCS-2024-77
2	Najeeba	NUM-BSCS-2024-61
3	Kashif Ali	NUM-BSCS-2024-29

Department of Computer Science
Namal University
Mianwali, Pakistan

Submission Date: 21 June 2026

REVISION HISTORY

Table 1: Revision History

Version	Date	Author	Description
V 1.0	15/05/2026	Asiya Batool	Final review completed. All sections proofread and Milestone 2 report submitted.
V 0.3	10/05/2026	Tayyab Shahzad	Business rules drafted and EERD finalized. Relationships and cardinalities verified.
V 0.2	05/05/2026	Tayyab Shahzad	Data dictionary completed. Attributes, data types, and constraints defined for all entities.
V 0.1	01/05/2026	Tayyab Shahzad	Initial draft of Milestone 2 started. Entities identified from system requirements.

Contents

Revision History	1
1 Project Overview	4
1.1 Introduction	4
1.2 Problem Statement	4
1.3 Project Objectives	5
1.4 GitHub Repository Link	5
2 Conceptual Database Design	6
2.1 Entities	6
2.2 Data Dictionary	6
2.2.1 Person	7
2.2.2 Freelancer	7
2.2.3 Client	7
2.2.4 Admin	8
2.2.5 Skills	8
2.2.6 Job_Posting	8
2.2.7 Proposal_Template	9
2.2.8 Reliability_Score	9
2.2.9 Freelancer_Analytics	9
2.2.10 Generated_Proposals	10
2.3 Business Rules	10
2.4 Entity Relationship Diagram	11
3 Logical Database Design	12
3.1 Relational Schema	12
3.2 Functional Dependencies	12
3.2.1 Person	12
3.2.2 Freelancer	12
3.2.3 Client	12
3.2.4 Admin	13
3.2.5 Skills	13
3.2.6 Freelancer_Skill_Map	13
3.2.7 Job_Posting	13

3.2.8	Job_Skill_Map	13
3.2.9	Proposal_Template	13
3.2.10	Reliability_Score	13
3.2.11	Freelancer_Analytics	13
3.2.12	Generated_Proposals	13
3.3	Normalization	13
3.3.1	First Normal Form (1NF)	13
3.3.2	Second Normal Form (2NF)	14
3.3.3	Third Normal Form (3NF)	15
3.3.4	Final 3NF Relations	16
4	Implementation	17
4.1	Language / Framework	17
4.2	Database Connectivity	17
4.3	Stored Procedures and Functions	17
4.3.1	Sample Stored Procedure	17
4.4	Database Views	18
4.5	Database Triggers	18
4.6	User Interfaces	18
5	Conclusion and Future Work	20
5.1	Summary of Work	20
5.2	Future Enhancements	20

CHAPTER 1: PROJECT OVERVIEW

1.1 INTRODUCTION

The freelance marketplace has become increasingly competitive, requiring freelancers to submit high-quality proposals quickly in order to secure projects. Platforms such as Upwork receive a large number of bids for every job posting, making response time and proposal quality critical factors in hiring decisions.

The Upwork Proposal Assistant is designed to streamline the proposal creation process by providing automated support for proposal generation, skills matching, template management, client evaluation, and performance tracking. The system utilizes a structured relational database to store and manage information related to freelancers, clients, job postings, skills, proposal templates, generated proposals, and analytical records.

By centralizing and organizing these resources, the system improves efficiency, reduces repetitive work, and enables freelancers to make data-driven decisions while applying for projects.

1.2 PROBLEM STATEMENT

Freelancers on online marketplaces often face several operational challenges that reduce productivity and affect proposal success rates. Some of the major problems include:

- **Manual Proposal Writing:** Freelancers spend significant time creating customized proposals for each job posting, resulting in delayed submissions and reduced competitiveness.
- **Lack of Centralized Data Management:** Important information such as reusable proposal content, client records, and freelancer skills is often scattered across multiple platforms and documents.
- **Limited Performance Analysis:** Most freelancers do not maintain structured records to analyze proposal acceptance rates, template effectiveness, or overall bidding performance.
- **Client Reliability Concerns:** Freelancers may unknowingly apply to projects posted by unreliable clients, leading to wasted time and resources.

These challenges highlight the need for a structured database-driven solution capable of managing information efficiently and supporting intelligent proposal generation.

1.3 PROJECT OBJECTIVES

The primary objective of this project is to design and implement a normalized, production-ready relational database for the Upwork Proposal Assistant that streamlines proposal generation, optimizes skills management, assesses client reliability, and tracks freelancer performance. To achieve this, the system establishes a Third Normal Form (3NF) normalized schema featuring a hierarchical user structure—separating users into freelancers, clients, and administrators—alongside multi-table associative skill mapping to properly handle complex data relationships. Additionally, the data layer systematically organizes reusable proposal templates while maintaining a comprehensive, chronological log of generation histories. To ensure data-driven decision-making, the architecture computes real-time freelancer performance metrics and statistical client reliability risk scores directly inside the relational layer. Ultimately, this structural optimization completely eradicates operational insertion, update, and deletion anomalies, creating an integrated ecosystem where freelancers can safely analyze client metrics and optimize their bidding strategy.

1.4 GITHUB REPOSITORY LINK

The project source code, database scripts, and implementation files are maintained through the following GitHub repository:

<https://github.com/iamtayyab-shahzad/Software-engineering-project>

CHAPTER 2: CONCEPTUAL DATABASE DESIGN

2.1 ENTITIES

The Upwork Proposal Assistant database consists of several entities that collectively support proposal generation, client management, freelancer management, skills mapping, and performance analysis. Each entity represents a distinct real-world object within the system.

Table 2.1: Entities of the System

Sr. No.	Entity Name	Description
1	Person	Stores common information for all system users.
2	Freelancer	Represents freelancers who generate and submit proposals.
3	Client	Represents clients who create job postings.
4	Admin	Represents administrators responsible for system management.
5	Skills	Stores technical skills used by freelancers and required by jobs.
6	Job_Posting	Stores job advertisements posted by clients.
7	Proposal_Template	Stores reusable proposal templates.
8	Reliability_Score	Stores client reliability and risk evaluation records.
9	Freelancer_Analytics	Stores proposal performance statistics of freelancers.
10	Generated_Proposals	Stores proposals generated by freelancers.

2.2 DATA DICTIONARY

The following data dictionary describes all entities, attributes, data types, constraints, and attribute descriptions used in the database design.

2.2.1 Person

Table 2.2: Person Data Dictionary

Sr.	Attribute	Data Type	Constraint	Description
1	person_id	INT	PK, NOT NULL	Unique identifier of a person.
2	f_name	VARCHAR(50)	NOT NULL	First name of the person.
3	l_name	VARCHAR(50)	NOT NULL	Last name of the person.
4	email	VARCHAR(100)	UNIQUE, NOT NULL	Email address of the user.
5	country	VARCHAR(50)	NULL	Country of residence.
6	upwork_profile_url	VARCHAR(255)	UNIQUE	Upwork profile URL.

2.2.2 Freelancer

Table 2.3: Freelancer Data Dictionary

Sr.	Attribute	Data Type	Constraint	Description
1	freelancer_id	INT	PK	Unique freelancer identifier.
2	password	VARCHAR(255)	NOT NULL	Encrypted login password.
3	hourly_rate	DECIMAL(6,2)	NOT NULL	Hourly service rate.
4	experience_years	INT	NOT NULL	Years of professional experience.

2.2.3 Client

Table 2.4: Client Data Dictionary

Sr.	Attribute	Data Type	Constraint	Description
1	client_id	INT	PK	Unique client identifier.
2	company_name	VARCHAR(100)	NULL	Client company name.
3	success_rate	DECIMAL(5,2)	NULL	Client hiring success rate.
4	feedback	TEXT	NULL	Client feedback information.

2.2.4 Admin

Table 2.5: Admin Data Dictionary

Sr.	Attribute	Data Type	Constraint	Description
1	admin_id	INT	PK	Unique administrator identifier.
2	role	VARCHAR(50)	NOT NULL	Administrative role.
3	joined_date	DATE	NOT NULL	Date of joining.
4	status	VARCHAR(20)	NOT NULL	Current account status.

2.2.5 Skills

Table 2.6: Skills Data Dictionary

Sr.	Attribute	Data Type	Constraint	Description
1	skill_id	INT	PK	Unique skill identifier.
2	skill_name	VARCHAR(100)	UNIQUE	Name of skill.
3	skill_level	VARCHAR(30)	NOT NULL	Skill proficiency level.

2.2.6 Job Posting

Table 2.7: Job Posting Data Dictionary

Sr.	Attribute	Data Type	Constraint	Description
1	job_id	INT	PK	Unique job identifier.
2	job_title	VARCHAR(150)	NOT NULL	Job title.
3	job_description	TEXT	NOT NULL	Detailed job description.
4	experience_level	VARCHAR(20)	NOT NULL	Required experience level.
5	job_type	VARCHAR(20)	NOT NULL	Job category.
6	project_duration	VARCHAR(50)	NULL	Estimated project duration.
7	budget_amount	DECIMAL(10,2)	NOT NULL	Project budget.
8	posted_date	TIMESTAMP	NOT NULL	Job posting date.
9	proposal_count	INT	DEFAULT 0	Number of proposals received.

2.2.7 Proposal_Template

Table 2.8: Proposal Template Data Dictionary

Sr.	Attribute	Data Type	Constraint	Description
1	template_id	INT	PK	Unique template identifier.
2	template_name	VARCHAR(100)	NOT NULL	Template name.
3	category	VARCHAR(50)	NOT NULL	Template category.
4	template_content	TEXT	NOT NULL	Proposal template text.
5	usage_count	INT	DEFAULT 0	Number of times used.
6	success_rate	DECIMAL(5,2)	NULL	Template success rate.
7	acceptance_rate	DECIMAL(5,2)	NULL	Template acceptance rate.

2.2.8 Reliability_Score

Table 2.9: Reliability Score Data Dictionary

Sr.	Attribute	Data Type	Constraint	Description
1	score_id	INT	PK	Unique score identifier.
2	reliability_score	DECIMAL(5,2)	NOT NULL	Reliability score value.
3	risk_level	VARCHAR(20)	NOT NULL	Risk category.
4	recommendation_level	VARCHAR(50)	NOT NULL	Recommendation category.
5	calculated_date	TIMESTAMP	NOT NULL	Date of score calculation.

2.2.9 Freelancer_Analytics

Table 2.10: Freelancer Analytics Data Dictionary

Sr.	Attribute	Data Type	Constraint	Description
1	analytics_id	INT	PK	Unique analytics identifier.
2	total_proposals	INT	DEFAULT 0	Total proposals submitted.
3	accepted_proposals	INT	DEFAULT 0	Accepted proposals.
4	rejected_proposals	INT	DEFAULT 0	Rejected proposals.
5	success_rate	DECIMAL(5,2)	DEFAULT 0	Freelancer success rate.

2.2.10 Generated Proposals

Table 2.11: Generated Proposals Data Dictionary

Sr.	Attribute	Data Type	Constraint	Description
1	proposal_id	INT	PK	Unique proposal identifier.
2	proposal_content	TEXT	NOT NULL	Generated proposal content.
3	generated_date	TIMESTAMP	NOT NULL	Proposal generation date.
4	proposal_status	VARCHAR(30)	NOT NULL	Current proposal status.
5	submission_date	TIMESTAMP	NULL	Proposal submission date.

2.3 BUSINESS RULES

The following business rules govern the operation of the database:

1. Every person must have a unique email address.
2. A freelancer must be associated with exactly one person.
3. A client must be associated with exactly one person.
4. An administrator must be associated with exactly one person.
5. A freelancer can possess multiple skills.
6. A skill can belong to multiple freelancers.
7. A client can create multiple job postings.
8. Each job posting belongs to exactly one client.
9. A job posting may require multiple skills.
10. A skill may be required by multiple job postings.
11. An administrator can create multiple proposal templates.
12. A freelancer can generate multiple proposals.
13. A proposal template can be used in multiple generated proposals.
14. Each generated proposal belongs to one freelancer.
15. Each generated proposal belongs to one client.
16. Each generated proposal belongs to one job posting.
17. A client can have multiple reliability score records.
18. A freelancer can have one analytics profile.

2.4 ENTITY RELATIONSHIP DIAGRAM

Figure 2.1 presents the Entity Relationship Diagram (ERD) of the Upwork Proposal Assistant system. The diagram illustrates entities, attributes, primary keys, foreign keys, and relationship cardinalities.

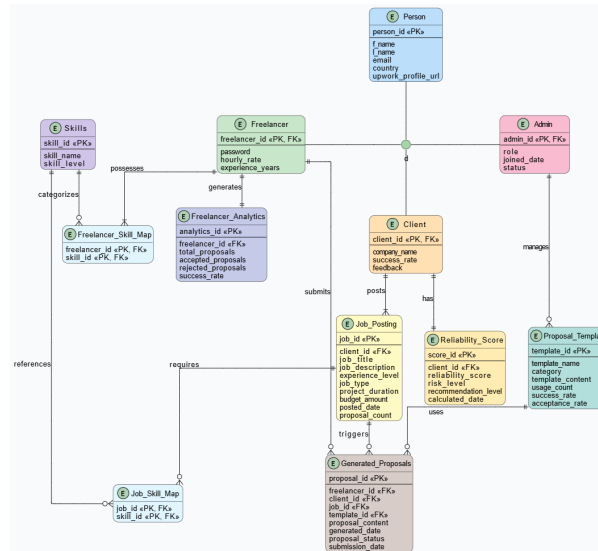


Figure 2.1: Entity Relationship Diagram of Upwork Proposal Assistant

CHAPTER 3: LOGICAL DATABASE DESIGN

3.1 RELATIONAL SCHEMA

The conceptual design was transformed into a set of normalized relational tables. The complete relational schema is shown in Figure 3.1. Primary Keys (PK) and Foreign Keys (FK) are clearly identified within the schema diagram.

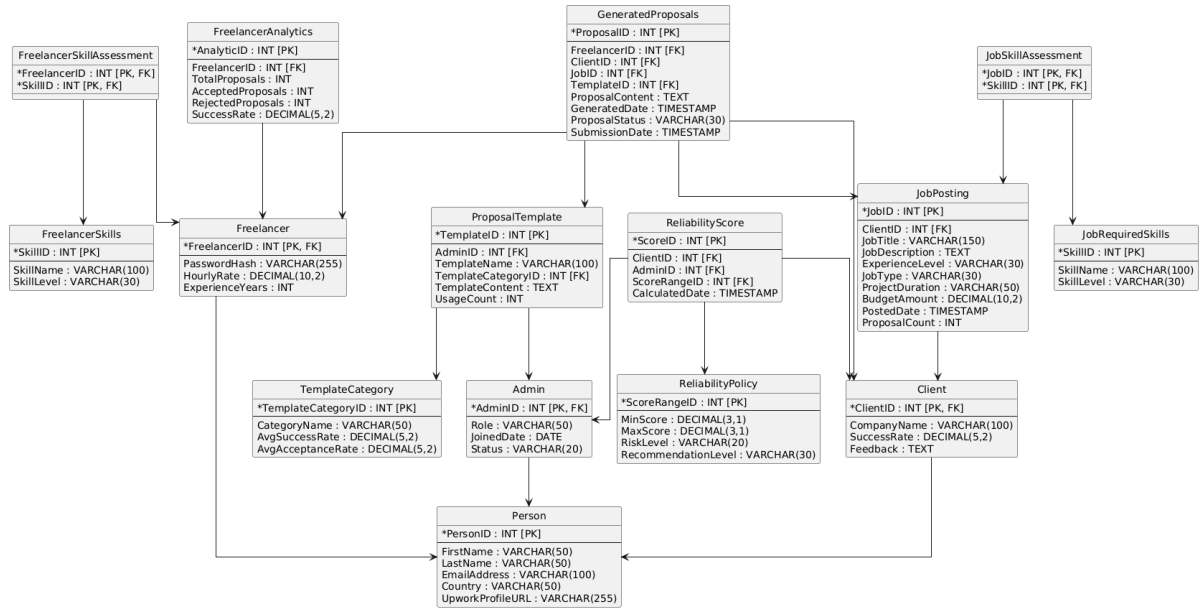


Figure 3.1: Relational schema of Upwork Proposal Assistant

3.2 FUNCTIONAL DEPENDENCIES

The following functional dependencies exist within the database design.

3.2.1 Person

$\text{person_id} \rightarrow \text{f_name}, \text{l_name}, \text{email}, \text{country}, \text{upwork_profile_url}$

3.2.2 Freelancer

$\text{freelancer_id} \rightarrow \text{password}, \text{hourly_rate}, \text{experience_years}$

3.2.3 Client

$\text{client_id} \rightarrow \text{company_name}, \text{success_rate}, \text{feedback}$

3.2.4 Admin

admin_id → role, joined_date, status

3.2.5 Skills

skill_id → skill_name, skill_level

3.2.6 Freelancer_Skill_Map

(freelancer_id, skill_id) → No Non-Key Attributes

3.2.7 Job_Posting

job_id → client_id, job_title, job_description, experience_level, job_type, project_duration, budget_amount, post

3.2.8 Job_Skill_Map

(job_id, skill_id) → No Non-Key Attributes

3.2.9 Proposal_Template

template_id → admin_id, template_name, category, template_content, usage_count, success_rate, acceptance_ra

3.2.10 Reliability_Score

score_id → client_id, reliability_score, risk_level, recommendation_level, calculated_date

3.2.11 Freelancer_Analytics

analytics_id → freelancer_id, total_proposals, accepted_proposals, rejected_proposals, success_rate

3.2.12 Generated_Proposals

proposal_id → freelancer_id, client_id, job_id, template_id, proposal_content, generated_date, proposal_status, s

3.3 NORMALIZATION

Normalization was performed to eliminate redundancy, improve data integrity, and prevent insertion, update, and deletion anomalies.

3.3.1 First Normal Form (1NF)

- All attributes contain atomic values.

- No repeating groups exist.
- Each record is uniquely identifiable through a primary key.
- Multi-valued attributes have been removed.

An example of an unnormalized structure is:

```
Freelancer(freelancer_id, skill_1, skill_2, skill_3)
```

This design violates 1NF because multiple values are stored within a single row. To resolve this issue, the design was decomposed into `Freelancer`, `Skills`, and `Freelancer_Skill_Map`, which stores each skill as an independent atomic record.

The structural transformation mapping out the atomic values for the first normal form is highlighted below in Figure 3.2.

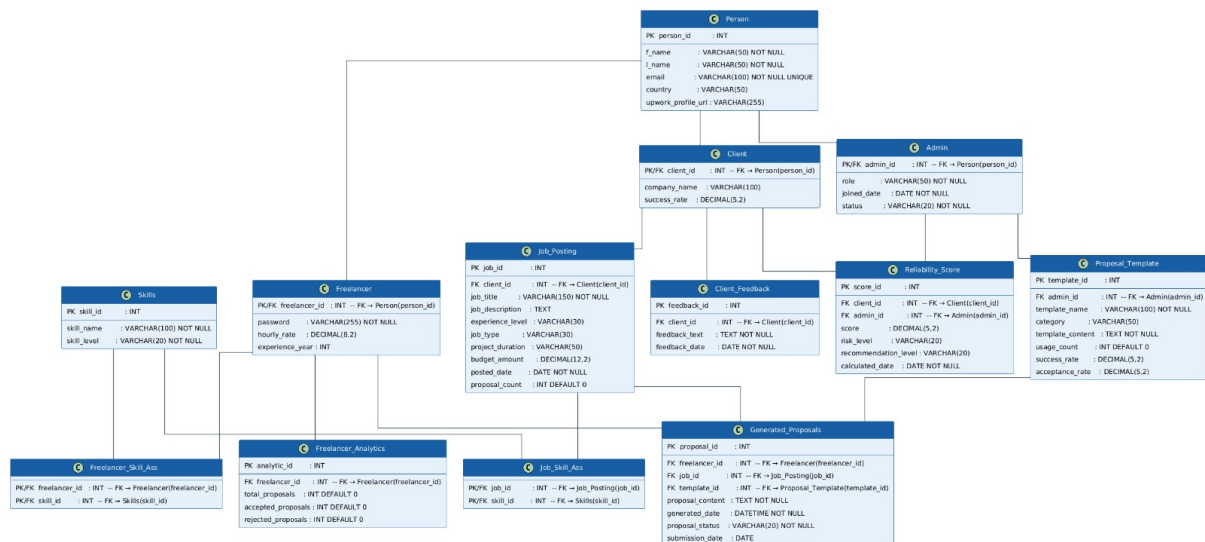


Figure 3.2: First Normal Form (1NF) Relational Mapping

3.3.2 Second Normal Form (2NF)

The database satisfies Second Normal Form because:

- All tables already satisfy 1NF.
- No partial dependencies exist.
- Every non-key attribute depends on the entire primary key.

For example, the composite key (`freelancer_id`, `skill_id`) in the `Freelancer_Skill_Map` table contains no non-key attributes that depend on only part of the composite key. Therefore, the relation satisfies 2NF. The full mapping schema showing the extraction of partial keys into secondary tables is displayed below in Figure 3.3.

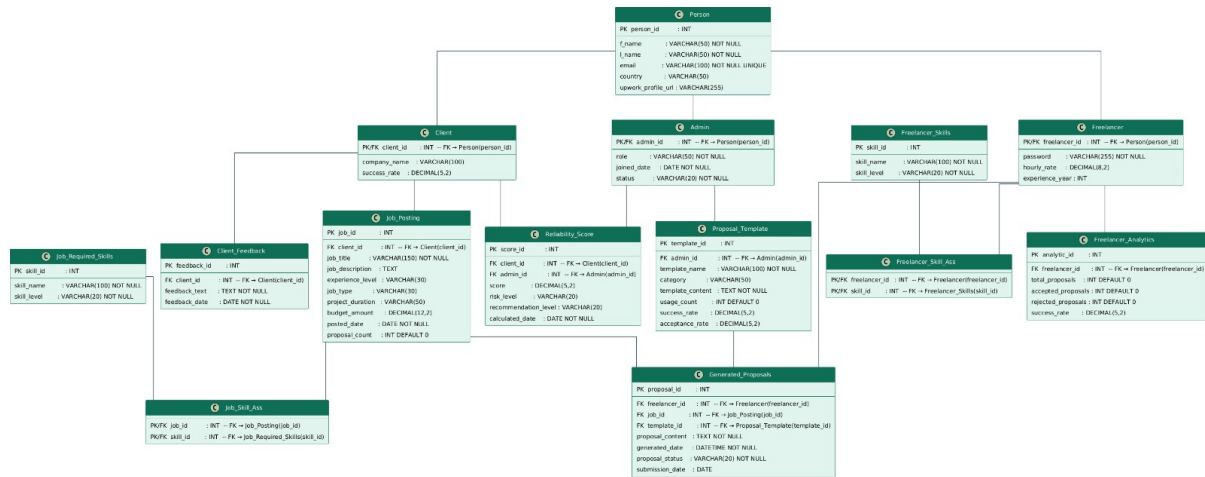


Figure 3.3: Second Normal Form (2NF) Dependency Diagram

3.3.3 Third Normal Form (3NF)

The database satisfies Third Normal Form because:

- All tables satisfy 2NF.
- No transitive dependencies exist.
- Non-key attributes depend only on their primary keys.
- Derived and analytical information is isolated into separate entities.

Examples include isolating reliability information into `Reliability_Score`, performance records into `Freelancer_Analytics`, and templates within `Proposal_Template`. This decoupled configuration is shown below in Figure 3.4.

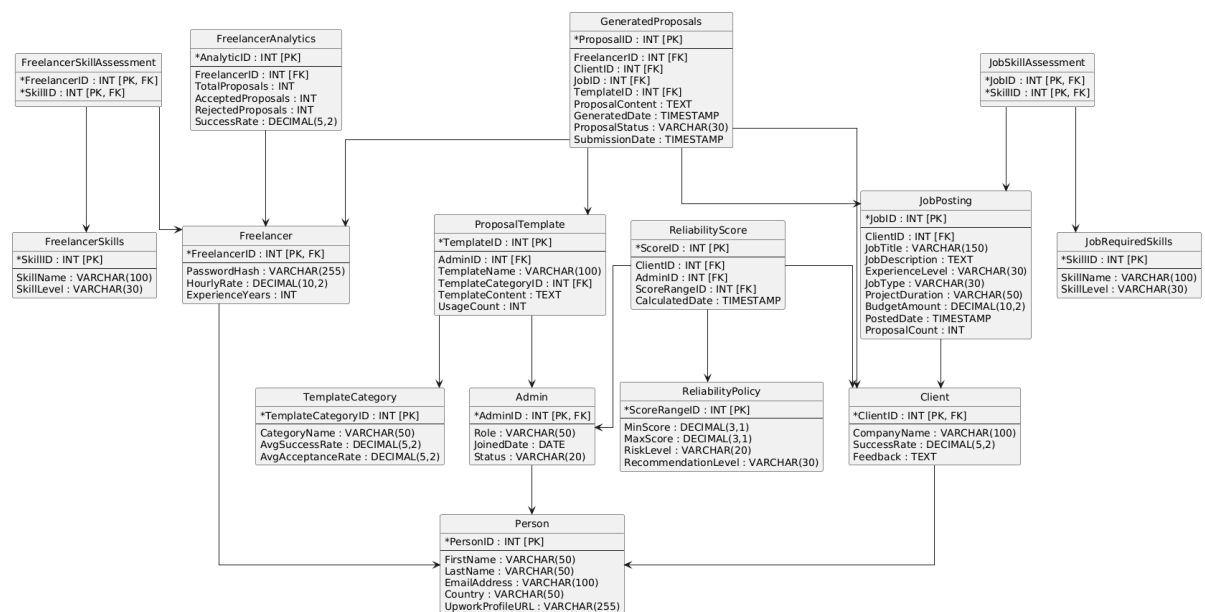


Figure 3.4: Third Normal Form (3NF) Transitive Resolution Schema

3.3.4 Final 3NF Relations

1. Person(person_id, f_name, l_name, email, country, upwork_profile_url)
2. Freelancer(freelancer_id, password, hourly_rate, experience_years)
3. Client(client_id, company_name, success_rate, feedback)
4. Admin(admin_id, role, joined_date, status)
5. Skills(skill_id, skill_name, skill_level)
6. Freelancer_Skill_Map(freelancer_id, skill_id)
7. Job_Posting(job_id, client_id, job_title, job_description, experience_level, job_type, project_duration, budget_amount, posted_date, proposal_count)
8. Job_Skill_Map(job_id, skill_id)
9. Proposal_Template(template_id, admin_id, template_name, category, template_content, usage_count, success_rate, acceptance_rate)
10. Reliability_Score(score_id, client_id, reliability_score, risk_level, recommendation_level, calculated_date)
11. Freelancer_Analytics(analytics_id, freelancer_id, total_proposals, accepted_proposals, rejected_proposals, success_rate)
12. Generated_Proposals(proposal_id, freelancer_id, client_id, job_id, template_id, proposal_content, generated_date, proposal_status, submission_date)

CHAPTER 4: IMPLEMENTATION

This chapter presents the implementation details of the Upwork Proposal Assistant system. It describes the technologies used, database connectivity mechanisms, database objects, and graphical user interfaces developed during implementation.

4.1 LANGUAGE / FRAMEWORK

The backend database layer is implemented using MySQL 8.0 with the InnoDB storage engine to ensure ACID-compliant transactions and reliable data management.

Table 4.1: Technologies Used

Technology	Purpose
Python / PyQt6	Desktop application development and graphical user interface.
MySQL 8.0	Relational Database Management System (RDBMS).
GitHub	Version control and repository management.

4.2 DATABASE CONNECTIVITY

The application communicates with the database using the `mysql-connector-python` driver. This driver provides secure database communication and enables execution of SQL queries from the application layer. Connection pooling and exception handling mechanisms are used to ensure efficient database access and maintain system reliability.

4.3 STORED PROCEDURES AND FUNCTIONS

Stored procedures are used to automate repetitive database operations and improve maintainability.

4.3.1 Sample Stored Procedure

```
DELIMITER $$

CREATE PROCEDURE UpdateSuccessRate (
    IN freelancerId INT
)
BEGIN
```

```

UPDATE Freelancer_Analytics
SET success_rate = (accepted_proposals / total_proposals) * 100
WHERE freelancer_id = freelancerId;
END$$

DELIMITER ;

```

4.4 DATABASE VIEWS

Database views simplify access to frequently used analytical information.

```

CREATE VIEW FreelancerPerformance AS
SELECT
    freelancer_id,
    total_proposals,
    accepted_proposals,
    success_rate
FROM Freelancer_Analytics;

```

4.5 DATABASE TRIGGERS

Triggers are used to automate updates whenever proposal records are inserted into the database.

```

CREATE TRIGGER UpdateProposalCount
AFTER INSERT ON Generated_Proposals
FOR EACH ROW
UPDATE Job_Posting
SET proposal_count = proposal_count + 1
WHERE job_id = NEW.job_id;

```

4.6 USER INTERFACES

The graphical user interface (GUI) of the Upwork Proposal Assistant is built using Python and the PyQt6 framework, providing a clean, responsive, and intuitive experience for freelancers and administrators. The frontend application interacts dynamically with the MySQL database backend via structured queries, executing transactional mutations and rendering real-time analytical records.

The user interface layer includes several core panels optimized for daily freelancing workflows:

- **Authentication Panel:** Handles multi-role secure login and session management for Freelancers, Clients, and Admins, validating cryptographic password strings against the database.

- **Job Scraping & Requirements Board:** Displays imported Upwork job postings along with their required tech stacks, mapped through the underlying `Job_Skill_Map` relation.
- **Automated Proposal Generator:** Allows users to select a specialized template, tweak dynamic parameters, and generate structured proposal text while simultaneously updating tracking analytics.
- **Client Reliability Evaluation Hub:** Renders the calculated statistical risk and client recommendation levels based on historical hiring success rates and analytical feedback loops.

The systematic architecture linking the desktop user interfaces directly through the database connectivity connector layer down to the underlying relational storage engine is illustrated below in Figure 4.1.

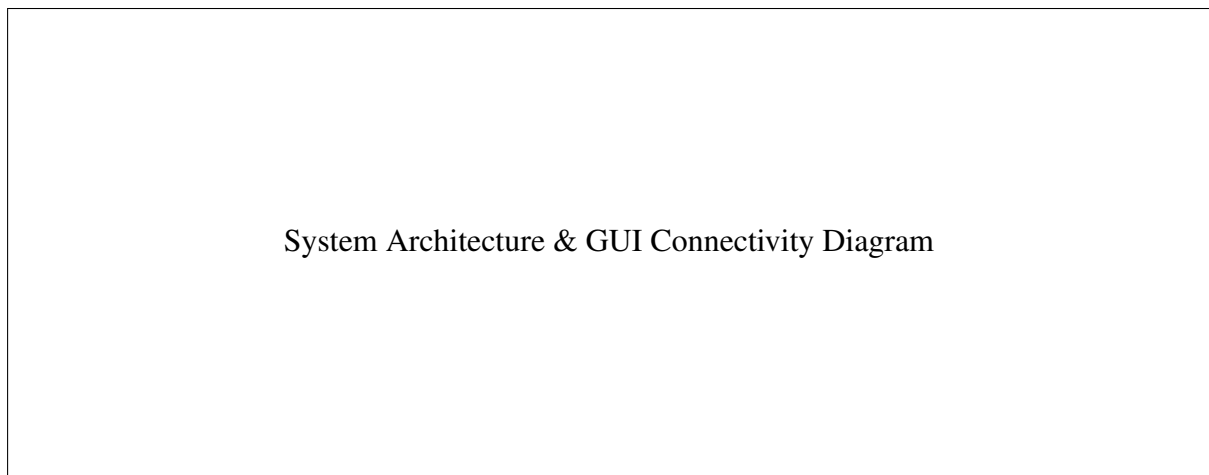


Figure 4.1: Upwork Proposal Assistant Desktop Application System Architecture

CHAPTER 5: CONCLUSION AND FUTURE WORK

5.1 SUMMARY OF WORK

The Upwork Proposal Assistant Database Design Document outlines a comprehensive, robust, and highly structured relational database system engineered explicitly to tackle the operational overhead faced by modern online freelancers. By transitioning away from decentralized, unstructured data files, this system provides a secure, single source of truth for managing user profiles, granular skill matrices, job post tracking, reusable templates, and historical proposal generation workflows.

Throughout the lifecycle of this architectural design, emphasis was placed on maintaining a highly normalized data model. Through rigorous decomposition steps, the schema was brought to Third Normal Form (3NF), entirely eliminating insertion, update, and deletion operational anomalies. Complex associative tables (`Freelancer_Skill_Map` and `Job_Skill_Map`) were introduced to properly manage multi-valued attributes and complex many-to-many relationships without data duplication.

Furthermore, performance and system encapsulation were prioritized at the data layer using advanced database objects. Automation was successfully offloaded to the database through the implementation of performance-tracking views, automated proposal counter triggers, and success rate calculation procedures.

5.2 FUTURE ENHANCEMENTS

While the current relational schema provides a production-grade foundational layer for desktop environments, future iterations of the Upwork Proposal Assistant will expand on architectural scalability and intelligent system integration:

- **Integration of Large Language Models (LLMs):** Future database revisions will include specialized tables for vector embeddings to store semantic representations of job descriptions and proposal templates, allowing the application layer to query the database using semantic vector similarity search.
- **Automated API Web Scraping:** Expand the data pipeline by integrating automated Upwork API cron-jobs to automatically populate `Job_Posting` records in real-time, completely removing manual tracking workflows.
- **Cloud Infrastructure Migration:** Transition the local MySQL storage engine to an enterprise-grade cloud-hosted relational database service (such as AWS RDS or Google Cloud SQL) featuring master-slave replication and connection pooling to support a cross-platform web and mobile ecosystem.

- **Advanced Risk Analytics Engine:** Enhance the `Reliability_Score` entity logic using machine learning classification models to calculate complex fraud indicators, ensuring freelancers can accurately evaluate client metrics before consuming valuable bidding tokens.